INT161 · Basic Backend Development

Exam Walkthrough

เริ่มทำข้อสอบจาก template ของอาจารย์ทีละขั้น: อะไรเป็นโครง อะไรพัง และลำดับที่ควรสร้าง

Open the Project | Database in VS Code | Offline Prep | Tour of the Template
Config and First Boot | Build Order | Test Without Postman | Error Middleware
Time Plan and Checklist | อภิธานศัพท์

01

Open the Project

อาจารย์ให้ **project template** มาพร้อมใบงาน คลาสนี้พาเดินตั้งแต่เปิดโปรเจกต์ถึง zip คนในทุกตัวอย่าง คือ Somchai Jaidee รหัส 67000000000 key AB12cd เป็นคนสมมติ

> ⚠ **ใส่ของคุณ**
> ที่ไหนก็ตามที่เห็น Somchai Jaidee, 67000000000 หรือ AB12cd ให้ใช้ชื่อ รหัส และ secret key จากใบงานของคุณเอง มีสามจุด: comment บรรทัดแรกของทุกไฟล์ ({student-id} - {student-name}), STUDENT_ID ใน app.js, และ name กับ secretKey ใน **route** student-info

1. แตก zip ของ template เช็คว่าไม่ได้โฟลเดอร์ซ้อนโฟลเดอร์ชื่อเดียวกัน
2. เปิดโฟลเดอร์ชั้นใน ที่มี package.json เป็นโปรเจกต์ใน IDE
3. นำเข้า database จากไฟล์ SQL ของคุณ หัวข้อ 2 บอกวิธีใน VS Code
4. รัน **npm** install ครั้งเดียวตอนที่ยังมีเน็ต แล้วปิดเน็ต
5. เปิดใบงานโหมด preview และอ่านให้ครบก่อนเขียนโค้ดสักบรรทัด

project folder

```
$ npm install
```

ไม่ต้องสนไฟล์เปล่าชื่อ npm และโฟลเดอร์ .idea ไม่ได้ทำอะไร

02

Database in VS Code

WebStorm มีเครื่องมือ database ในตัว VS Code ไม่มี จึงต้องนำเข้าไฟล์ SQL ด้วยวิธีอื่น MySQL และ Workbench ติดตั้งไว้แล้วตั้งแต่คลาส 4 ใช้ Workbench ก่อน เพราะไม่ต้องใช้ command line

วิธีที่ 1: MySQL Workbench

1. เปิด **connection** ในเครื่องแล้วใส่ password ของ root
2. File, Open SQL Script เลือก init-database.sql
3. กดปุ่มสายฟ้าเพื่อรันทั้งสคริปต์
4. refresh แผง Schemas จะเห็น mobile_distributor พร้อมสอง table

ระหว่างนี้ให้ดูชื่อ table และคอลัมน์จริง หน้านี้สมมติว่าเป็น brands กับ products และไฟล์ SQL ของคุณคือคำตอบที่ถูก

วิธีที่ 2: command line

คำสั่งในใบงาน mysql -u root -p < file.sql ใช้ไม่ได้ใน PowerShell เพราะไม่มีตัวดำเนินการ < ให้เลือกอย่างใดอย่างหนึ่งต่อไปนี้แทน

PowerShell

```
# run it through cmd, which understands <
> cmd /c "mysql -u root -p < database\init-database.sql"
# or pipe the file in
> Get-Content database\init-database.sql | mysql -u root -p
# or log in, then source it with forward slashes
> mysql -u root -p
mysql> source C:/Users/you/exam/database/init-database.sql
```

ถ้า Windows บอกว่าไม่รู้จัก mysql ให้เพิ่มโฟลเดอร์ bin ของ MySQL ใน **PATH** เช่น C:\Program Files\MySQL\MySQL **Server** 8.0\bin เลขเวอร์ชันในพาธอาจต่างกันในเครื่องคุณ แล้วเปิด terminal ใหม่

วิธีที่ 3: ใน VS Code (ไม่บังคับ)

extension เช่น SQLTools พร้อม MySQL **driver** ช่วยรัน **query** ใน editor ได้ มีก็ดีแต่ไม่จำเป็น ต้องติดตั้งก่อนสอบ เพราะข้อสอบเป็น offline

เช็คว่าสำเร็จ

```mysql
mysql> USE mobile_distributor; SHOW TABLES;
mysql> SELECT COUNT(*) FROM products;
```

> ⚠ service ต้องทำงานอยู่
> ถ้า connection ถูกปฏิเสธ แปลว่า MySQL ไม่ทำงาน กด Win+R พิมพ์ services.msc หา MySQL80 (ชื่ออาจต่างตามการติดตั้ง) แล้วกด Start ส่วน password ผิดใน db-config.js จะได้ ER_ACCESS_DENIED_ERROR

03

Offline Prep

ข้อสอบเป็น offline และรัน npm install ได้ครั้งเดียว ทุกอย่างที่ต้องดาวน์โหลดต้องพร้อมก่อนปิดเน็ต ไล่รายการนี้ก่อนวันสอบ แล้วไล่อีกครั้งตอนเริ่มสอบ

1. รัน **node** -v **Express** 5 ต้องใช้ Node 18 ขึ้นไป
2. ติดตั้ง extension ของ VS Code ที่จะใช้: **REST Client** และ SQLTools ถ้าต้องการ ติดตั้งตอน offline ไม่ได้
3. File, Open Folder แล้วเลือกโฟลเดอร์ที่มี package.json terminal จะเริ่มในที่ที่ถูกต้อง และ npm จะไม่บ่นว่าหา package.json ไม่เจอ
4. รัน npm install ครั้งเดียวตอนที่ยังมีเน็ต ดูว่ามีโฟลเดอร์ node_modules
5. เปิด File, Auto Save ไฟล์ที่ลืม save คือสาเหตุคลาสสิกที่ทำให้การแก้ดูเหมือนไม่ได้ผล
6. เช็คว่า Workbench ยังเชื่อมต่อได้ และไฟล์ SQL นำเข้าได้สะอาด
7. ปิดเน็ตแล้วลองทำเต็มรอบบน template ตั้งแต่หัวข้อ 4 จนถึง zip การเปิดเน็ตกลับระหว่างสอบถือว่าทุจริต

04

Tour of the Template

ชื่อโฟลเดอร์ลงท้ายด้วย $id ไม่ต้องแก้ import ทุกตัวใน template ใช้ชื่อนี้อยู่แล้ว การเปลี่ยนชื่อมีแต่เสี่ยงพัง

app.js

เริ่ม server mount router ตัวเดียวที่ "/" และมี error handler ว่าง ๆ คุณต้องเขียนใหม่

configs-$id/db-config.js

MySQL pool ทุกค่าเป็นค่าแทน

routes-$id/route.js

router กลางตัวเดียวที่ตอบข้อความแทน copy ไปทำทีละ resource

services-$id/service.js

เรียก repository กับ mapper มีบั๊กตอนเจอ null

repositories-$id/repository.js

SQL กลางที่ใช้ชื่อ table แทน update กับ delete ไม่เคยให้ 404

mappers-$id/

api-response.js ห่อ JSON ส่วน entity-dto-mapper.js เขียนไว้สำหรับอีก database

middlewares-$id/pagination-middleware.js

อ่าน page กับ limit คำนวณ offset ผิด

test-service.js

ไฟล์ลองเรียก service โดยไม่ผ่าน HTTP path ที่ import ใช้ไม่ได้

template เป็นแค่โครง ยังไม่มีอะไรตอบตามใบงาน คุณ copy route, service และ **repository** กลางไปทำทีละ **resource** เหมือนคลาส 6: brand-route.js, brand-repository.js, product-route.js และอื่น ๆ เก็บไฟล์ต้นฉบับไว้เป็นตัวอ้างอิง

05

Config and First Boot

แก้เล็ก ๆ สามจุดแล้วรัน ข้อแรก package.json ไม่มี start script แต่ใบงานให้รัน npm start ให้เพิ่มเข้าไป

package.json

```
"scripts": {
  "start": "node src/app.js"
},
```

ข้อสอง กรอก **pool** ใน db-config.js database คือ mobile_distributor

configs-$id/db-config.js

```
// 67000000000 - Somchai Jaidee
import mysql from 'mysql2/promise'
const db = mysql.createPool({
    host: 'localhost',
    user: 'root',
    password: 'your-mysql-password',
    database: 'mobile_distributor',
    dateStrings: ['DATE'], // release_date stays '2023-09-22', no timezone shift
})
export default db
```

ข้อสาม เขียน app.js ใหม่ ซึ่งสร้าง **endpoint** ยืนยันตัวตนด้วย คะแนนแรก 10 คะแนน error handler อยู่ก่อน listen และหลัง **router**

src/app.js

```
// 67000000000 - Somchai Jaidee
import express from 'express'
import brandRouter from './routes-$id/brand-route.js'
import productRouter from './routes-$id/product-route.js'
import { errorHandler } from './middlewares-$id/error-middleware.js'
const STUDENT_ID = '67000000000'
const app = express()
app.use(express.json())
app.get(`/api/${STUDENT_ID}/student-info`, (req, res) => {
    res.status(200).json({
        success: true,
        data: { studentId: STUDENT_ID, name: 'Somchai Jaidee', secretKey: 'AB12cd' },
        timestamp: new Date().toISOString(),
    })
})
app.use(`/api/${STUDENT_ID}/brands`, brandRouter)
app.use(`/api/${STUDENT_ID}/products`, productRouter)
app.use(errorHandler) // after the routers, before listen
app.listen(3000, () => console.log('listening on port 3000'))
```

import ชี้ไปยังไฟล์ที่จะสร้างในหัวข้อถัดไป สร้างไฟล์เปล่าไว้ก่อน หรือ comment บรรทัดนั้นไว้จนกว่าจะถึง

สร้าง error handler ก่อน เพราะ app.js import มัน

middlewares-$id/error-middleware.js

```
// 67000000000 - Somchai Jaidee
export class AppError extends Error {
    constructor(statusCode, message) {
        super(message)
        this.statusCode = statusCode
    }
}
export const errorHandler = (err, req, res, next) => {
    const status = err.code === 'ER_DUP_ENTRY' ? 409 : err.statusCode || 500
    res.status(status).json({
        success: false,
        error: { message: err.message },
        timestamp: new Date().toISOString(),
    })
}
```

boot and test

```
$ npm start
# second terminal, or the browser
$ curl localhost:3000/api/67000000000/student-info
```

> **หยุดเช็ค**
>
> ควรเห็น success true รหัส ชื่อ และ key ของคุณ ถ้าเห็นแล้ว ข้อ 1 เสร็จ ใช้ลำดับนี้ไปตลอดข้อสอบ: ทำเสร็จทีละ endpoint ทดสอบ แล้วค่อยไปต่อ

06

Build Order

สร้างตามลำดับนี้ เล็กและแน่นอนก่อน ก้อนคะแนนใหญ่ตอนที่จับ pattern ได้แล้ว

1. student-info

app.js
10 คะแนน ตอบ 200 พร้อมชื่อและ key ใช้เวลาห้านาที

2. Brands

brand-repository, brand-route
GET รายการและ GET หนึ่งรายการ 404 เมื่อไม่มี

3. Products list

pagination, repository, mapper, service, route
30 คะแนน มากที่สุด ใช้ summary DTO และ block pagination

4. Product detail

repository (JOIN), mapper
200 หรือ 404 brandName มาจาก join

5. POST

repository, route
201 เมื่อบันทึกได้ 409 เมื่อ SKU ซ้ำ

6. PUT and DELETE

repository, route
PUT 200, DELETE 204 ไม่มี body ทั้งคู่ 404 เมื่อไม่มี

ขั้นที่ 2: Brands

copy repository.js เป็น brand-repository.js และ route.js เป็น brand-route.js Brands ไม่ต้องมี service หรือ **mapper** เพราะคอลัมน์ตรงกับ **response** อยู่แล้ว route จึงเรียก repository ตรง ๆ ทางลัดนี้ใช้ได้กับรายการเล็กแบบนี้

repositories-$id/brand-repository.js

```
// 67000000000 - Somchai Jaidee
import db from '../configs-$id/db-config.js'
const FIND_ALL_SQL = 'SELECT * FROM brands'
const FIND_ONE_SQL = 'SELECT * FROM brands WHERE id = ?'
export async function findAll() {
    const [rows] = await db.query(FIND_ALL_SQL)
    return rows
}
export async function findOne(id) {
    const [rows] = await db.query(FIND_ONE_SQL, [id])
    return rows[0] ?? null // null, not a throw: the route decides it is a 404
}
```

routes-$id/brand-route.js

```
// 67000000000 - Somchai Jaidee
import * as repo from '../repositories-$id/brand-repository.js'
import { apiResponse } from '../mappers-$id/api-response.js'
import { AppError } from '../middlewares-$id/error-middleware.js'
import express from 'express'
const router = express.Router()
router.get('/', async (req, res) => {
    await apiResponse(res, await repo.findAll())
})
router.get('/:id', async (req, res) => {
    const brand = await repo.findOne(req.params.id)
    if (!brand) throw new AppError(404, `Brand ${req.params.id} not found`)
    await apiResponse(res, brand)
})
export default router
```

route ใน template ไม่ได้ส่ง **timestamp** ที่ใบงานต้องการ แก้ที่ api-response.js ครั้งเดียว ทุก endpoint ก็ได้

mappers-$id/api-response.js

```
// in both success() and paginate(), add one line after `data`:
        timestamp: new Date().toISOString(),
```

ทดสอบ: GET /brands, GET /brands/1, GET /brands/99999 อันสุดท้ายต้องได้ 404

ขั้นที่ 3: รายการ Products

ไฟล์ใน template สามไฟล์ใช้ไม่ได้กับขั้นนี้ **pagination middleware** คูณด้วย **page size** ที่เป็น string ดิบ และคืน null เมื่อไม่มี page แต่ใบงานให้ค่าเริ่มต้นเป็นหน้า 1 ให้แทนที่ทั้งไฟล์

middlewares-$id/pagination-middleware.js

```
// 67000000000 - Somchai Jaidee
export const queryParser = () => (req, res, next) => {
    const page = Math.max(parseInt(req.query.page) || 1, 1)
    const limit = Math.min(Math.max(parseInt(req.query.limit) || 10, 1), 100)
    req.pagination = { page, limit, offset: (page - 1) * limit } // times limit, not the raw stri
    next()
}
```

mapper เขียนไว้สำหรับอีก database (productCode, MSRP) เขียนใหม่ให้ตรงกับคอลัมน์ของเรา detail ต่อยอดจาก **summary** จึงเก็บ field ไว้ที่เดียว

mappers-$id/entity-dto-mapper.js

```
// 67000000000 - Somchai Jaidee
export default class EntityDtoMapper {
    static toSummaryDTO(e) {
        return {
            id: e.id, sku: e.sku, name: e.name, brandId: e.brand_id,
            category: e.category, price: Number(e.price), status: e.status,
        }
    }
    static toDetailDTO(e) {
        return {
            ...this.toSummaryDTO(e),
            brandName: e.brand_name, ramGb: e.ram_gb, storageGb: e.storage_gb,
            color: e.color, stockQuantity: e.stock_quantity, releaseDate: e.release_date,
        }
    }
    static map(entity, viewType) {
        return viewType == 'summary' ? this.toSummaryDTO(entity) : this.toDetailDTO(entity)
    }
    static maps(entities, viewType) {
        const { totalItems } = entities
        const out = entities.map(item => this.map(item, viewType))
        out.totalItems = totalItems // apiResponse reads the total from here
        return out
    }
}
```

repository ต้องนับแถวด้วย ไม่งั้น response บอก totalPages ไม่ได้ template เขียน query นับไว้แต่ไม่เคยเรียก ไฟล์นี้มี join ของ detail การเช็ค SKU และ logic ของ 404 สำหรับขั้นถัดไปด้วย

repositories-$id/product-repository.js

```
// 67000000000 - Somchai Jaidee
import db from '../configs-$id/db-config.js'
const FIND_ALL_SQL = 'SELECT * FROM products LIMIT ? OFFSET ?'
const COUNT_SQL = 'SELECT COUNT(*) AS total FROM products'
const FIND_ONE_SQL = `SELECT p.*, b.name AS brand_name
    FROM products p JOIN brands b ON b.id = p.brand_id WHERE p.id = ?`
const FIND_SKU_SQL = 'SELECT id FROM products WHERE sku = ?'
const CREATE_SQL = 'INSERT INTO products SET ?'
const UPDATE_SQL = 'UPDATE products SET ? WHERE id = ?'
const REMOVE_SQL = 'DELETE FROM products WHERE id = ?'
export async function findAll({ limit, offset }) {
    const [rows] = await db.query(FIND_ALL_SQL, [limit, offset])
    const [[{ total }]] = await db.query(COUNT_SQL)
    rows.totalItems = total
    return rows
}
export async function findOne(id) {
    const [rows] = await db.query(FIND_ONE_SQL, [id])
    return rows[0] ?? null
}
export async function create(data) {
    const [dup] = await db.query(FIND_SKU_SQL, [data.sku])
    if (dup.length) return null // SKU already exists, the route answers 409
    const [result] = await db.query(CREATE_SQL, [data])
    return findOne(result.insertId)
}
export async function update(id, data) {
    const [result] = await db.query(UPDATE_SQL, [data, id])
    return result.affectedRows ? findOne(id) : null
}
export async function remove(id) {
    const [result] = await db.query(REMOVE_SQL, [id])
    return result.affectedRows > 0
}
```

services-$id/product-service.js

```
// 67000000000 - Somchai Jaidee
import * as repo from '../repositories-$id/product-repository.js'
import mapper from '../mappers-$id/entity-dto-mapper.js'
const detail = (row) => row && mapper.map(row, 'full') // null stays null
export async function getAllObjects(pagination) {
    return mapper.maps(await repo.findAll(pagination), 'summary')
}
export async function getObject(id) { return detail(await repo.findOne(id)) }
export async function createObject(data) { return detail(await repo.create(data)) }
export async function updateObject(id, data) { return detail(await repo.update(id, data)) }
export async function removeObject(id) { return repo.remove(id) }
```

routes-$id/product-route.js

```
// 67000000000 - Somchai Jaidee
import * as service from '../services-$id/product-service.js'
import { apiResponse } from '../mappers-$id/api-response.js'
import { queryParser } from '../middlewares-$id/pagination-middleware.js'
import { AppError } from '../middlewares-$id/error-middleware.js'
import httpStatus from 'http-status'
import express from 'express'
const router = express.Router()
const notFound = (id) => new AppError(404, `Product ${id} not found`)
router.get('/', queryParser(), async (req, res) => {
    const data = await service.getAllObjects(req.pagination)
    await apiResponse(res, data, req.pagination)
})
router.get('/:id', async (req, res) => {
    const data = await service.getObject(req.params.id)
    if (!data) throw notFound(req.params.id)
    await apiResponse(res, data)
})
router.post('/', async (req, res) => {
    const data = await service.createObject(req.body)
    if (!data) throw new AppError(409, 'SKU already exists')
    await apiResponse(res, data, null, httpStatus.CREATED)
})
router.put('/:id', async (req, res) => {
    const data = await service.updateObject(req.params.id, req.body)
    if (!data) throw notFound(req.params.id)
    await apiResponse(res, data)
})
router.delete('/:id', async (req, res) => {
    if (!await service.removeObject(req.params.id)) throw notFound(req.params.id)
    res.status(httpStatus.NO_CONTENT).end()
})
export default router
```

ทดสอบรายการด้วย page=1&limit=10 แล้ว page=2 เทียบ total, totalPages และ hasNextPage กับตัวอย่างในใบงาน

ขั้นที่ 4 ถึง 6: Detail, POST, PUT, DELETE

ไฟล์ข้างบนครอบคลุมแล้ว ขั้นเหล่านี้จึงเน้นทดสอบ เช็คแต่ละข้อกับ status ของมัน:

- GET /products/1 ตอบ 200 พร้อม brandName GET /products/99999 ตอบ 404
- POST **body** จากใบงาน ครั้งแรกได้ 201 ครั้งที่สอง sku เดิมได้ 409 body เป็น **snake_case** และคอลัมน์ก็เช่นกัน จึงใส่ใน INSERT ได้เลย
- PUT /products/:id พร้อม field ที่เปลี่ยนได้ 200 id ที่ไม่มีได้ 404
- DELETE ตอบ 204 และ body ว่าง DELETE id เดิมซ้ำเป็น 404

ถ้าอยากลองแต่ละ layer โดยไม่ผ่าน **HTTP** ให้เขียน test-service.js ใหม่ template import office-service.js ซึ่งไม่มีไฟล์นี้

src/test-service.js

```
import * as service from './services-$id/product-service.js'
console.log(await service.getObject(1))
console.log(await service.getObject(99999)) // null, no crash
process.exit() // the pool keeps Node alive otherwise
```

> ⚠ **ชื่อที่สมมติ**
> ตอนเขียนหน้านี้ไม่มีไฟล์ SQL ของคุณ ชื่อ table brands, products และชื่อคอลัมน์มาจาก field ใน response ของใบงาน ถ้า IDE แสดงชื่ออื่น ให้แก้ SQL ให้ตรง

07

Test Without Postman

browser ส่งได้แค่ GET แต่คุณต้องทดสอบ POST, PUT และ DELETE ด้วย และจะไม่มี **Postman** extension REST Client (id humao.rest-client) เปลี่ยนไฟล์ข้อความธรรมดาให้เป็น **request** ที่กดได้ ติดตั้งก่อนสอบ

สร้าง test.http ที่ root ของโปรเจกต์ จะมีลิงก์ Send Request เหนือทุก request กดแล้ว response เปิดข้างไฟล์ บรรทัดที่ขึ้นต้นด้วย ### คั่นแต่ละ request

test.http

```
@base = http://localhost:3000/api/67000000000
### student-info: 200
GET {{base}}/student-info
### brands: 200
GET {{base}}/brands
### brand 99999: 404
GET {{base}}/brands/99999
### products page 1: 200 with pagination
GET {{base}}/products?page=1&limit=10
### product detail: 200, then 404 for id 99999
GET {{base}}/products/1
### create: 201 the first time, 409 the second
POST {{base}}/products
Content-Type: application/json
{ "sku": "TEST-SKU-999", "name": "Test Phone 2026", "brand_id": 1, "category": "FLAGSHIP",
  "price": 29900.00, "ram_gb": 8, "storage_gb": 256, "color": "Black",
  "stock_quantity": 10, "status": "AVAILABLE", "release_date": "2026-01-01" }
### update: 200, 404 for a missing id
PUT {{base}}/products/1
Content-Type: application/json
{ "price": 31900.00 }
### delete: 204, 404 the second time
DELETE {{base}}/products/76
```

- แทนรหัสใน @base ด้วยของคุณ product id 76 ใน DELETE เป็นตัวอย่าง ให้ลบสินค้าที่ POST สร้าง โดยใช้ id ที่ POST ตอบกลับมา
- แต่ละ request บอก status ที่ควรได้ เทียบกับบรรทัด status ด้านบนของ response
- ไม่มี extension? ใช้ **curl**.exe ใน Windows PowerShell 5 คำว่า curl เฉย ๆ เป็น alias ของอีกคำสั่ง จึงต้องพิมพ์ .exe การใส่ quote ให้ **JSON** ใน command line ยุ่งยาก ใช้ไฟล์ .http ดีกว่า
- test.http เป็นของคุณ จะเอาออกจาก zip หรือเก็บไว้ก็ได้ ไม่มีผลต่อการตรวจ

08

Error Middleware

template จบ app.js ด้วย handler ว่าง ๆ app.use((err, req, res, next) => {}) มันจับทุก error แล้วไม่ตอบอะไร request จึงค้างจน timeout คุณแทนที่ไปแล้วในหัวข้อ 5 นี่คือวิธีที่ชิ้นส่วนเชื่อมกัน

- route **throw** new AppError(404, message) Express 5 ส่ง error จาก **async** route ไปที่ handler เอง จึงไม่ต้องมี try catch และไม่ต้อง next(err)
- handler อ่าน statusCode จาก error ถ้าเป็น duplicate entry จาก database (ER_DUP_ENTRY) จะเป็น 409 ถ้าไม่มี status จะเป็น 500
- handler ต้องลงทะเบียนหลัง router ทุกตัว handler ที่ลงก่อนจะไม่เห็น error ของ router

> **status แต่ละตัวมาจากไหน**
> 404 throw ใน route เมื่อ repository คืน null หรือ false 409 throw เมื่อ create คืน null หรือมาจาก database 201 และ 204 กำหนดในเส้นทางที่สำเร็จ ไม่มีอะไรอื่นต้องมี status

09

Time Plan and Checklist

สองชั่วโมงพอถ้าไม่ติดนาน แผนคร่าว ๆ:

- 15 นาที: เปิดโปรเจกต์, database, config, app.js, student-info
- 15 นาที: brands
- 30 นาที: รายการ products คะแนนเยอะสุด อย่ารีบ
- 35 นาที: detail, POST, PUT, DELETE
- 25 นาที: รันตัวตรวจ แก้ข้อที่ไม่ผ่าน zip เก็บเวลาช่วงนี้ไว้ อย่าเอาไปใช้กับ endpoint ที่ยาก

ถ้ามีอะไรพัง ดูตรงนี้ก่อน:

`Server starts, request never answers`

error ถูกกลืน app.use(errorHandler) อยู่ใน app.js ไหม อยู่ก่อน listen หรือเปล่า

`Cannot find module`

path ที่ import ผิด ชื่อโฟลเดอร์ยังมี $id ต่อท้าย ให้พิมพ์ตรงตามเดิม

`Table does not exist`

ชื่อ table ในหน้านี้ (brands, products) เป็นการเดา ดูชื่อจริงใน data source ของ IDE แล้วแก้ให้ตรง

`Price comes back as a string`

MySQL ส่ง DECIMAL เป็นข้อความ mapper ครอบด้วย Number()

`Page 2 shows the wrong rows`

offset ใช้ string ดิบ ให้ใช้ pagination middleware ในหน้านี้

ก่อน zip ไล่เช็คดังนี้:

1. comment บรรทัดแรกของทุกไฟล์ที่แตะมีรหัสและชื่อของคุณเอง
2. app.js มี STUDENT_ID, ชื่อ และ secret key ของคุณ ทุก route อยู่ใต้ /api/รหัสของคุณ
3. node tests/test-runner.mjs รหัสของคุณ ผ่านทุกข้อเท่าที่ทำได้
4. ลบ node_modules, zip เฉพาะโฟลเดอร์งาน ตั้งชื่อ รหัสของคุณ_exam.zip

> **ข้อม**
>
> ลองทำทั้งหมดหนึ่งรอบ แบบ offline พร้อมจับเวลา ก่อนสอบจริง รอบแรกเสียเวลากับการตั้งค่าเสมอ รอบสองไม่เสีย

10

อภิธานศัพท์

async
: เริ่มงานที่ช้า แล้วทำงานอื่นต่อ และถูกเรียกกลับเมื่อเสร็จ แทนที่จะรอ

body
: ข้อมูลที่ request พกมาหลัง header สำหรับ API มักเป็น JSON

browser
: client ที่คนส่วนใหญ่ใช้ เช่น Chrome, Edge, Safari

Client
: ฝั่งที่ร้องขอ มักเป็น browser แต่เป็นโปรแกรมหรืออุปกรณ์อะไรก็ได้

connection

ช่องทางที่เปิดอยู่ระหว่างโปรแกรมกับ database ต้องมีก่อนส่ง SQL ได้

curl

client แบบ command line ส่ง HTTP method ใดก็ได้ ใช้ทดสอบ POST และ PUT ได้ ใส่ -i เพื่อดู header ของ response

driver

library ที่รู้วิธีคุยกับ database ชนิดหนึ่ง สำหรับ MySQL ใน Node คือ mysql2

endpoint

method กับ path หนึ่งคู่ที่ API ตอบ เช่น GET /api/subjects

Express

web framework ขนาดเล็กของ Node.js เพิ่ม routing และ middleware บน http module ที่มีมาให้

HTTP

HyperText Transfer Protocol: ภาษาข้อความธรรมดาที่ browser กับ web server ใช้คุยกัน ฝั่งหนึ่งส่ง request อีกฝั่งส่ง response

JSON

JavaScript Object Notation: รูปแบบข้อมูลที่เป็นข้อความ คนและโปรแกรมอ่านได้

limit

จำนวนรายการต่อหนึ่งหน้า client ส่งมาเป็น limit และ server กำหนดเพดาน

mapper

class ที่แปลง entity เป็น DTO เพื่อให้การแปลงอยู่ที่เดียวและใช้ซ้ำได้

middleware

ฟังก์ชัน (req, res, next) ที่ทำงานระหว่าง request กับ route handler แก้ request ตอบเอง หรือส่งต่อได้

node

runtime ที่รัน JavaScript นอก browser ทำให้ใช้ภาษาเดียวกันเขียน back end ได้

npm

Node Package Manager: ติดตั้ง library เช่น express ลงใน node_modules

page size

จำนวนรายการต่อหนึ่งหน้า client ส่งมาเป็น limit และ server กำหนดเพดาน

pagination

ส่งรายการขนาดใหญ่ทีละหน้า เพื่อให้ response เล็กและเร็ว

PATH

ค่าตั้งของ Windows ที่ระบุโฟลเดอร์ซึ่งใช้ค้นหาคำสั่งที่พิมพ์ ถ้า bin ของ MySQL ไม่อยู่ในนี้ จะไม่รู้จักคำสั่ง mysql

pool

การเชื่อมต่อที่เปิดค้างไว้ให้ใช้ร่วมกัน request ยืมไปใช้แล้วคืน database จึงไม่ถูกท่วม

Postman

เครื่องมือส่ง HTTP method ใดก็ได้ไปที่ API ใช้ทดสอบก่อนที่ front end จะเสร็จ

project template

โปรเจกต์ตั้งต้นที่บันทึกไว้ โปรเจกต์ใหม่ที่สร้างจากมันจะมีโฟลเดอร์และ code พื้นฐานให้แล้ว

query

ส่วนของ URL หลัง ? เช่น page=2&size=10 ใช้กรองหรือแบ่งหน้ารายการ

repository

layer ที่อ่านเขียน database ไม่มี business rule และไม่มี HTTP

request

สิ่งที่ client ส่งมา: method, URL, header และบางครั้งมี body

resource

สิ่งที่ API ให้เข้าถึง เช่น product หรือ customer ในวิชานี้ตรงกับ table ใน database

response

สิ่งที่ server ส่งกลับ: status code, header และ body

REST

แนวทางออกแบบ web API: resource ตั้งชื่อด้วย URL และเลือก action ด้วย HTTP method

route

การจับคู่ HTTP method และ URL path กับฟังก์ชันที่จัดการ

router

การจับคู่ HTTP method และ URL path กับฟังก์ชันที่จัดการ

Server

โปรแกรม (หรือเครื่องที่รันมัน) ที่รอรับ request แล้วตอบกลับ

service

layer ที่เก็บ business rule เรียก repository และไม่รู้เรื่อง HTTP

snake_case

คำต่อกันด้วย underscore เช่น user_name ใช้บ่อยกับคอลัมน์ใน database ไม่ใช้กับ URI

summary

DTO สองขนาด: summary มีไม่กี่ field สำหรับรายการ detail มีครบสำหรับรายการเดียว

throw

โยน error ออกมา โยนได้เฉพาะ object ของ Error

timestamp

ค่าวันที่และเวลา คอลัมน์อย่าง last_update ใส่เวลาปัจจุบันให้เองได้